

Codigo de alto nivel
A + B

Codigo de nivel intermediario

```
mov eax, [a] -- Leva o valor de A para eax
add eax, [b] -- soma o valor de eax com o valor de b, armazenando a soma em eax
```

Como os dados estão representados na memoria?
R: É um vetor de bytes

```
{
Execução exemplo:
nasm -f elf64 hello.asm && ld hello.o -o hello.x
```

```
executa no terminal:
/hello.x
}
```

C
A = B; copia valor da posição de B para a posição de A.

asmble
lea rsi, [str0la]; Move o valor e o endereço.; (lea) precisa de []

sem [] = -> &
com [] = Conteudo

little endian lsb ;Menos significante byte
bigger endian msb ;Mais signnificante byte

gdb ;Debug do linux

gdb -q [nome do arquivo]; executa e não mostra todos os parametros do gdb

gdb q ; sair

gdb Ctrl + L; limpa

gdb run ;Executa o programa e termina ele

gdb break (fim) ; para o programa no criterio de parada no caso fim

gdb c ; continua o programa apos o break

gdb x &v1; manda imprimir o conteudo da variavel v1; & para chamar variavel; imprime todos os bytes, imprimindo o padrão a partir do endereço

gdb x / ;/[variavel] define o formato que vai ser imprimindo

gdb x /xb &v1 ;imprime formato em hexa
; se colocar um foramto que não funciona ele mantem o foramto anterior

gdb x /db &v1 ;imprime formato decimal

gdb x /cb &v1 ;imprime caracter byte

gdb x /xb 0x402000 ;imprime um byte da posição de memoria

gdb x /xh 0x402001 ;imprime dois bytes // // // //

gdb x /lcb 0x402004 /imprime um caracter byte da posição

gdb x /8cb &v4 ;imprime a quantidade de caracteres

gdb x /s &v4 ;imprime string até encontrar um 0, incluindo lixo de memoria e outros

gdb x /xh &v5 ;Hexa o endereço a esquerda é o mais significativo

gdb x /xb &v5 ;Saida 0x34 o menos significante a direita

14/10/22

Na aula 2, codigo a02e01.xb

```

v5 possui 2 bits contendo 34 12
v6 possui 2 bits contendo 'a' &
v7 // 2 bits // 'a' 'b'
v8 // 4 bits // ['a' 'b'] [c_&_]
v9 // 4 bits // [78 56 34 12]

```

x/w &v1

-> 0x57565555

; vai do mais significativo ao menos

(gdb) x /28xb &v1

```

0x402000: 0x55 0x55 0x56 0x57 0x61 0x55 0x68 0x65
0x402008: 0x6c 0x6c 0x6f 0x0d 0x0a 0x24 0x34 0x12
0x402010: 0x61 0x00 0x61 0x62 0x61 0x62 0x63 0x00
0x402018: 0x78 0x56 0x34 0x12

```

;utilizando um numero apresenta todos os bytes, sendo eles reservados em 3 espaços de memoria

x /th &v10

0x40201c: 0011100010000000

; numero de bytes da variavel 10

x /dh &v10

0x40201c: 14464

; diferente da String em que foi transformado em vetor para manter os dados

; o numero não transforma em vetor, logo trunca e perde os dados

atividade pratica

todo conteudo de memoria:

(gdb) x /92xb &v1

```

0x402000: 0x45 0x45 0x46 0x47 0x78 0x45 0x58 0x45
0x402008: 0x68 0x65 0x6c 0x6c 0x6f 0x0d 0x0a 0x24
0x402010: 0x4f 0x4c 0x41 0x0d 0x0a 0x30 0x87 0x19
0x402018: 0x7a 0x00 0x5a 0x78 0x5a 0x78 0x59 0x00
0x402020: 0x5a 0x78 0x59 0x77 0x48 0x39 0x56 0x07
0x402028: 0x01 0x80 0xca 0x39 0xff 0xff 0xe7 0x89
0x402030: 0x04 0x23 0xc7 0x8a 0xa0 0x00 0x00 0x00
0x402038: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x402040: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x402048: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x402050: 0x00 0x00 0x00 0x00 0x03 0x00 0x01 0x00
0x402058: 0x00 0x10 0x40 0x00

```

para saber espaço de memoria x 0x402001

v1 espaço de memoria 0x402001

v2 // // // 0x402002 ~ 0x402003

v3 // // // 0x402004 e 0x402005 ; 'x' na tabela ascii 0x78

v3b // // // 0x402006 e 0x402007 ; 'X' na tabela ascii 0x58

v4 // // // 0x402008 ~ 0x402009 ; conteudo de 16 bits

v4b // // // 0x402010 ~ 0x402015

v5 // // // 0x402016 ~ 0x402017

v6 // // // 0x402018 ~ 0x402019

v7 // // // 0x402020 ~ 0x402021

v10b ; dq nesse caso não pode ser exibido como dd dw db, porque não possui registrador que comporte o seu tamanho ou maior

Aula 3

arquivo a02e01.asm

p /x &[ação] ; printa o valor, não necessita mostrar tamanho como x /xg

(gdb) x /xg &num

0x402000: 0x7766554433221101

```
(gdb) p /x $rax
$1 = 0x7766554433221101
(gdb) p /x $eax
$2 = 0x33221101
(gdb) p /x $ax
$3 = 0x1101
(gdb) p /x $al
$4 = 0x1 ;byte mais significativo
(gdb) p /x $sendoski ;registrador não existe cuidado com nome do registrador
$5 = 0x0
(gdb) p /x $ah
$6 = 0x11 ;segundo byte mas significativo
```

```
(gdb) p /x $rax $rbx
A syntax error in expression, near `rbx'.
; p não printa varios registradores

; info registradores faz isso!
; primeira coluna em hexadecimal e segunda decimal
(gdb) i r rax rbx eax ; mostra valores dos registradores ....
rax          0x7766554433221101  8603657889541918977
rbx          0x0                0
eax          0x33221101         857870593
```

```
(gdb) i r ;mostra todos os registradores!
rax          0x7766554433221101  8603657889541918977
rbx          0x0                0
rcx          0x0                0
rdx          0x0                0
rsi          0x0                0
rdi          0x0                0
rbp          0x0                0x0
rsp          0x7fffffffdfa0     0x7fffffffdfa0
r8           0x0                0
r9           0x0                0
r10          0x0                0
r11          0x0                0
r12          0x0                0
r13          0x0                0
r14          0x0                0
r15          0x0                0
rip          0x401008           0x401008 <fim>
eflags      0x202              [ IF ]
cs          0x33                51
ss          0x2b                43
ds          0x0                0
es          0x0                0
fs          0x0                0
gs          0x0                0
```

```
arquivo a03e02.asm
...
mov r8, [num]
mov r9d, [num]
...
(gdb) i r r8 r9 r9d
r8          0x7766554433221101  8603657889541918977
r9          0x33221101         857870593
r9d         0x33221101         857870593
; valor de passado de r8 para r9d mas so parcial
; logo chamar r9 mostrarar apenas os valores de parciais de r9d
; tamanho de registrador é documento!

arquivo a03e03.asm
```

Aula 4: Variáveis Não Inicializadas e Estrutura de Programas
arquivos : a04e01.asm

Teste 1:
(gdb) b _start
Ponto de parada 1 at 0x401000

```
(gdb) b fim
Ponto de parada 2 at 0x401023
(gdb) r
Starting program: /home/vinicius/Área de Trabalho/Linguagem de Montagem/Execicios em Aula/Aula 4/a04e01.x
```

```
Breakpoint 1, 0x0000000000401000 in _start ()
(gdb) x /xb &v1
0x402000: 0x61
(gdb) x /xw &v2
0x402001: 0x65646362
(gdb) x /xb &n1
0x402008: 0x00
(gdb) x /xw &n2
0x402009: 0x00000000
(gdb) x /xb &n3
0x40200d: 0x00
(gdb) c
Continuing.
```

```
Breakpoint 2, 0x0000000000401023 in fim ()
(gdb) x /xb &n1
0x402008: 0x61
(gdb) x /xw &n2
0x402009: 0x65646362
(gdb) x /xb &n3
0x40200d: 0x61
(gdb) q
```

```
Teste 2:
(gdb) b isadora
Ponto de parada 1 at 0x401023
(gdb) b fim
Ponto de parada 2 at 0x401033
Starting program: /home/vinicius/Área de Trabalho/Linguagem de Montagem/Execicios em Aula/Aula 4/a04e01.x
```

```
Breakpoint 1, 0x0000000000401023 in isadora ()
(gdb) x /xb &n1
0x40200c: 0x61
(gdb) x /xw &n2
0x40200d: 0x65646362
(gdb) x /xb &n3
0x402011: 0x61
(gdb) c
Continuing.
```

```
Breakpoint 2, 0x0000000000401033 in fim ()
(gdb) x /xb &n1
0x40200c: 0xff
(gdb) x /xw &n2
0x40200d: 0x65ffffff
(gdb) x /xb &n3
0x402011: 0x61
(gdb) Quit
```

arquivos : a04e02.asm

```
(gdb) b fim
Ponto de parada 1 at 0x401032
(gdb) r
Starting program: /home/vinicius/Área de Trabalho/Linguagem de Montagem/Execicios em Aula/Aula 4/a04e02.x
```

```
Breakpoint 1, 0x0000000000401032 in fim ()
(gdb) p /x $al
$1 = 0x61
(gdb) x /xb &v1
0x402000: 0x61
(gdb) p /x $r8
$2 = 0x402000
(gdb) x /xb 0x402000
0x402000: 0x61
(gdb) x /xw &v2
```

```

0x402001: 0x65646362
(gdb) i r rbx rcx
rbx          0x65646362      1701077858
rcx          0x65646362      1701077858
(gdb) p /x $r9
$3 = 0x402001
(gdb) p /x $r10
$4 = 0x401000
(gdb) p /x $r11
$5 = 0x401032
(gdb) x /xb &v1
0x402000: 0x61
(gdb) x /xb &n1
0x402008: 0x00
(gdb) p /x $rsp
$6 = 0x7fffffffdfa0
(gdb) x /xg 0x401032
0x401032 <fim>: 0x0000bf00000003cb8
                                ; 3cb8 é o valor de 60

```

arquivos : a04e03.asm

arquivos : a04e04.asm

Starting program: /home/vinicius/Área de Trabalho/Liguagem de Montagem/Execicios em Aula/Aula 4/a04e04.x

```

Breakpoint 1, 0x0000000000401007 in fim ()
(gdb) x /xg &v1
0x402000: 0x0000000000000064
(gdb) x /xg &v2
0x402008: 0xfffffffffffff9c
(gdb) x /xd &v1
0x402000: 100
(gdb) x /xd &v2
0x402008: -100
(gdb) x /xg &v3
0x402010: 0x00000000ffffff9c
(gdb) x /dw &v3
0x402010: -100
(gdb) x /dg &v3
0x402010: 4294967196
(gdb) p /d $edx
$1 = -100
(gdb) p /d $rdx
$2 = 4294967196
    ;tamanho é documento! o numero é o mesmo, apenas a exibição esta equivocada

```

arquivos : a04e05.asm

```

(gdb) i r eax rax rcx
eax          0xffffffff9c      -100
rax          0xffffffff9c      4294967196
rcx          0xfffffffffffff9c -100

```

Aula 5

Arquivo : a05e01.asm

```

(gdb) b b1
Ponto de parada 1 at 0x401014
(gdb) b b2
Ponto de parada 2 at 0x401018
(gdb) b fim
Ponto de parada 3 at 0x40101c

```

```

Breakpoint 1, 0x0000000000401014 in b1 ()
(gdb) i r eax ebx ecx edx
eax          0xa              10
ebx          0x14             20
ecx          0x1e             30
edx          0xfffffffffe     -2

```

```
(gdb) c
Continuing.
```

```
Breakpoint 2, 0x0000000000401018 in b2 ()
```

```
(gdb) i r eax ebx ecx edx
```

```
eax      0xa      10
ebx      0x1e     30
ecx      0x1e     30
edx      0x8      8
```

```
(gdb) c
Continuing.
```

```
Breakpoint 3, 0x000000000040101c in fim ()
```

```
(gdb) i r eax ebx ecx edx
```

```
eax      0xffffffff6  -10
ebx      0x1e     30
ecx      0x14     20
edx      0x8      8
```

```
(gdb) c
Continuing.
```

```
[Inferior 1 (process 3182) exited normally]
```

```
(gdb) q
```

```
exemplo 2
```

```
Breakpoint 2, 0x000000000040102f in b3 ()
```

```
(gdb) p $r8d
```

```
$1 = 90
```

```
(gdb) i r r8d eax
```

```
r8d      0x5a      90
eax      0xffffffff6  -10
```

```
(gdb) q
```

```
arquivo a05e02.asm
```

```
Breakpoint 1, 0x0000000000401008 in overflow ()
```

```
(gdb) i r ax bx
```

```
ax      0x7fff     32767
bx      0xffff     -1
```

```
(gdb) p /u $bx
```

```
$1 = 65535
```

```
(gdb) p /t $ax
```

```
$2 = 111111111111111
```

```
(gdb) p /t $bx
```

```
$3 = 111111111111111
```

```
(gdb) i r eflags
```

```
eflags  0x202     [ IF ]
```

```
(gdb) c
```

```
Continuing.
```

```
Breakpoint 2, 0x000000000040100c in carry ()
```

```
(gdb) p /d $ax
```

```
$4 = -32768
```

```
(gdb) p /t $ax
```

```
$5 = 10000000000000000
```

```
(gdb) i r eflags
```

```
eflags  0xa96     [ PF AF SF IF OF ]
```

```
(gdb) c
```

```
Continuing.
```

```
Breakpoint 3, 0x0000000000401010 in fim ()
```

```
(gdb) i r eflags
```

```
eflags  0x257     [ CF PF AF ZF IF ]
```

```
(gdb) q
```

```
arquivo a05e03.asm
```

```
aula 6
```

```
arquivo a06e02.asm
```

```
gdb -q a06e02.x
Reading symbols from a06e02.x...
(No debugging symbols found in a06e02.x)
(gdb) b _start
Ponto de parada 1 at 0x401000
(gdb) b leitura
Ponto de parada 2 at 0x401020
(gdb) r
Starting program: /home/vinicius/Área de Trabalho/Liguagem de Montagem/Execicios em Aula/Aula 6/a06e02.x

Breakpoint 1, 0x0000000000401000 in _start ()
(gdb) x /d &fileHandle
0x402020:  0
(gdb) c
Continuing.

Breakpoint 2, 0x0000000000401020 in leitura ()
(gdb) c
Continuing.
kenji
batata
vodka
l[Inferior 1 (process 3458) exited normally]
(gdb) c
The program is not being run.
```